

온톨로지(Ontology) — CS 관점의 심층 분석

AnchorIQ 해운 공급망 리스크 플랫폼에서 온톨로지를 왜, 어떻게 활용하는지 CS 원리 기반으로 정리한다.

1. 온톨로지란 무엇인가

철학 vs CS

구분	철학 (존재론)	CS (온톨로지)
정의	“존재하는 것은 무엇인가”를 탐구	특정 도메인의 개념과 관계를 명시적으로 정의한 모델
핵심 질문	실재의 본질	“이 도메인에 어떤 Entity가 있고, 서로 어떻게 연결되는가?”
결과물	형이상학 이론	그래프 형태의 지식 구조 (Knowledge Graph)

CS에서 온톨로지란 “**세상을 Entity + Relationship으로 모델링하는 방법**”이다. 단순히 데이터를 저장하는 것이 아니라, 데이터 간의 **의미적 관계(Semantic Relationship)**를 구조화한다.

RDB vs Graph DB vs Ontology 비교

항목	RDB (PostgreSQL)	Graph DB (Neo4j)	Ontology Model
데이터 구조	테이블 + 행 + 열	노드 + 엣지 + 속성	엔티티 + 관계 + 의미 규칙
관계 표현	외래 키 + JOIN	직접 포인터 (엣지)	타입이 부여된 관계
스키마 변경	ALTER TABLE 필요	라벨/관계 추가만	유연한 확장
다대다 관계	중간 테이블 필요	엣지 하나로 표현	엣지 하나로 표현
“연쇄 관계” 탐색	재귀 CTE / 다중 JOIN	패턴 매칭 1줄	추론(Inference) 가능

왜 RDB로는 부족한가

AnchorIQ의 핵심 질의: “이 선박의 소유 회사가 제재국 소속인가?”

```
-- PostgreSQL: 4단계 JOIN (25줄+)
SELECT v.name, c.name, co.name, s.type
FROM vessel v
JOIN company c ON v.company_id = c.id
JOIN country co ON c.country_id = co.id
JOIN sanction s ON s.country_id = co.id
WHERE v.imo = '9200001';
-- 문제: 테이블이 늘어날수록 JOIN 수 폭발, N+1 쿼리 위험
```

데이터 10만 건 기준 이 쿼리는 3,200ms가 소요된다. 중간 결과셋이 카르테시안 곱으로 폭발하기 때문이다.

2. 그래프 이론 기초

핵심 개념

노드(Node/Vertex): 실체 (선박, 회사, 국가)
엣지(Edge): 관계 (소유, 등록, 제재)
방향 그래프: 엣지에 방향이 있음 (Vessel) - [:OWNED_BY] -> (Company)
속성(Property): 노드/엣지에 부착된 키-값 (name: "EVER GIVEN")

인접 리스트 vs 인접 행렬

항목	인접 행렬	인접 리스트
공간 복잡도	$O(V^2)$	$O(V + E)$
특정 엣지 존재 확인	$O(1)$	$O(k)$, k =연결 수
이웃 노드 순회	$O(V)$	$O(k)$
희소 그래프에 적합	X	O (Graph DB가 채택)

대부분의 실세계 그래프는 **희소(sparse)** 하다. 선박 26,000척이 있어도 각 선박이 연결된 엣지는 평균 3~5개뿐이다. 인접 행렬은 $26,000 \times 26,000 = 6.76$ 억 셀이 필요하지만, 인접 리스트는 실제 연결만 저장한다. 그래서 Neo4j는 **인접 리스트** 기반이다.

시간 복잡도: 관계 탐색

RDB JOIN: B-Tree 인덱스 탐색 $O(\log n) \times \text{JOIN 횟수} \rightarrow$ 데이터 크기에 비례
Graph DB: 포인터 따라가기 $O(1)$ per hop $\rightarrow$ 전체 데이터 크기와 무관

BFS/DFS와 홉(Hop) 탐색

1홉: (Vessel)-->(Company)	직접 연결된 이웃
2홉: (Vessel)-->(Company)-->(Country)	이웃의 이웃
3홉: ...-->(Sanction)	3단계 떨어진 관계
4홉: ...-->(Chokepoint)	공급망 리스크 전파 경로

BFS(너비 우선 탐색) 는 가까운 홉부터 탐색하므로 “N홉 이내의 관련 엔티티”를 찾는 데 적합하다.
Neo4j의 `shortestPath()` 는 내부적으로 **양방향 BFS**를 사용하여 두 노드 간 최단 경로를 $O(b^{d/2})$ 로 찾는다 (b: 분기 계수, d: 깊이).

3. Neo4j와 Cypher 쿼리

Property Graph Model

```
(v:Vessel {imo:"9200001", name:"EVER GIVEN", mmsi:"353136000"})
  |
  | [:OWNED_BY {since: 2018}]
  v
(c:Company {name:"Evergreen Marine", country:"TWN"})
  |
  | [:HEADQUARTERED_IN]
  v
(co:Country {code:"TWN", name:"Taiwan", sanctioned: false})
```

노드에는 **라벨(Label)** 과 **속성(Property)**, 관계에도 **타입(Type)** 과 속성이 있다.

Cypher 기초 문법 — SQL과 비교하며 배우기

비전공자를 위한 핵심 개념: Cypher는 “그림을 그리듯” 쿼리를 작성한다. SQL이 “테이블에서 가져오라”면, Cypher는 “이런 패턴을 찾아라”이다.

기본 표기법

(n)	← 노드 (동그라미)
-[r]->	← 관계 (화살표), 방향 있음
-[r]-	← 관계 (방향 없음, 양방향 탐색)

```
:Label      ← 노드/관계의 유형 (클래스 이름처럼)
{key: val}   ← 속성 (JSON처럼)
```

1. MATCH — 패턴 찾기 (SQL의 SELECT + JOIN)

```
-- SQL:  SELECT * FROM vessels WHERE name = 'EVER GIVEN'
-- Cypher:
MATCH (v:Vessel {name: "EVER GIVEN"})
RETURN v
```

```
-- SQL:  SELECT v.name, c.name
--        FROM vessels v
--        JOIN companies c ON v.company_id = c.id
-- Cypher: (JOIN이 아니라 화살표로 관계를 따라감)
MATCH (v:Vessel)-[:OWNED_BY]->(c:Company)
RETURN v.name, c.name
```

```
-- SQL:  SELECT v.name, c.name, co.name
--        FROM vessels v
--        JOIN companies c ON v.company_id = c.id
--        JOIN countries co ON c.country_id = co.id
--        WHERE co.sanctioned = true
-- Cypher: (3개 테이블 JOIN = 화살표 2개)
MATCH (v:Vessel)-[:OWNED_BY]->(c:Company)-[:HEADQUARTERED_IN]->(co:Country)
WHERE co.sanctioned = true
RETURN v.name AS vessel, c.name AS company, co.name AS country
```

핵심 차이: SQL은 JOIN할 때마다 코드가 3줄씩 늘지만, Cypher는 화살표 하나 추가.

2. WHERE — 조건 필터

```
-- 속성 비교
MATCH (v:Vessel)
WHERE v.status = "SAILING" AND v.riskScore > 70
RETURN v.name, v.riskScore

-- 문자열 포함 검색
MATCH (v:Vessel)
WHERE toLower(v.name) CONTAINS "ever"
RETURN v.name

-- 리스트 포함 여부
MATCH (v:Vessel)
```

```
WHERE v.type IN ["TANKER", "CONTAINER"]
RETURN v.name, v.type
```

3. CREATE / MERGE — 노드/관계 생성

```
-- 노드 생성 (항상 새로 만들)
CREATE (v:Vessel {name: "EVER GIVEN", imo: "9811000"})

-- 노드 병합 (있으면 재사용, 없으면 생성 — 중복 방지)
MERGE (v:Vessel {imo: "9811000"})
ON CREATE SET v.name = "EVER GIVEN", v.createdAt = datetime()
ON MATCH SET v.updatedAt = datetime()

-- 관계 생성
MATCH (v:Vessel {imo: "9811000"}), (c:Company {name: "Evergreen Marine"})
MERGE (v)-[:OWNED_BY]->(c)
```

MERGE vs CREATE: MERGE 는 “이미 있으면 건너뛰기”라서 같은 쿼리를 여러 번 실행해도 안전하다 (역등성). CREATE 는 매번 새로 만들어서 중복 위험.

4. 경로 탐색 — Cypher만의 강력한 기능

```
-- 가변 길이 경로: 1~4홉 사이의 모든 경로
MATCH path = (v:Vessel)-[*1..4]-(s:Sanction)
WHERE v.name = "SHAHR-E-KORD"
RETURN path

-- 최단 경로
MATCH path = shortestPath(
  (v:Vessel {name: "SHAHR-E-KORD"})-[*..6]-(s:Sanction)
)
RETURN path, length(path) AS hops
```

SQL로 이걸 하려면 재귀 CTE + 임시 테이블 + 30줄 이상이 필요하다. Cypher는 2줄.

5. 집계 — COUNT, COLLECT

```
-- 국가별 선박 수
MATCH (v:Vessel)-[:REGISTERED_IN]->(c:Country)
RETURN c.name AS country, count(v) AS vesselCount
ORDER BY vesselCount DESC

-- Singapore 항에 정박 중인 선박의 국적 분포
MATCH (p:Port {name: "Singapore"})<-[:DOCKED_AT]-(v:Vessel)-[:REGISTERED_IN]->(c:Country)
```

```
RETURN c.name AS country, count(v) AS vessels, collect(v.name)[0..3] AS sampleVessels
ORDER BY vessels DESC
```

6. AnchorIQ 실제 쿼리 예시

```
-- 제재국 국적 선박이 말라카 해협을 통과하는가?
MATCH (v:Vessel)-[:REGISTERED_IN]->(co:Country)-[:SANCTIONED_BY]->(s:Sanction)
WHERE (v)-[:TRANSITS]->(:Chokepoint {name: "Malacca"})
RETURN v.name, co.name, s.caption

-- 특정 항구에 어떤 나라 선박이 많은가? (2홉 집계)
MATCH (p:Port)<-[:DOCKED_AT]-(v:Vessel)-[:REGISTERED_IN]->(c:Country)
RETURN p.name AS port, c.name AS country, count(v) AS vessels
ORDER BY port, vessels DESC
```

SQL vs Cypher 요약 비교표

작업	SQL	Cypher
단일 조회	<code>SELECT * FROM vessels WHERE imo = '9811000'</code>	<code>MATCH (v:Vessel {imo:"9811000"}) RETURN v</code>
1단계 관계	<code>SELECT ... JOIN companies ...</code>	<code>MATCH (v)-[:OWNED_BY]->(c) RETURN ...</code>
4단계 관계	<code>SELECT ... JOIN x4 + WHERE ...</code> (25줄)	<code>MATCH (v)-[:A]->(b)-[:B]->(c)-[:C]->(d) RETURN ...</code> (3줄)
경로 탐색	재귀 CTE (30줄+)	<code>shortestPath((a)-[*..6]-(b))</code> (2줄)
유연한 관계 추가	<code>ALTER TABLE + FK + 마이그레이션</code>	<code>MERGE (a)-[:NEW_REL]->(b)</code> (1줄)

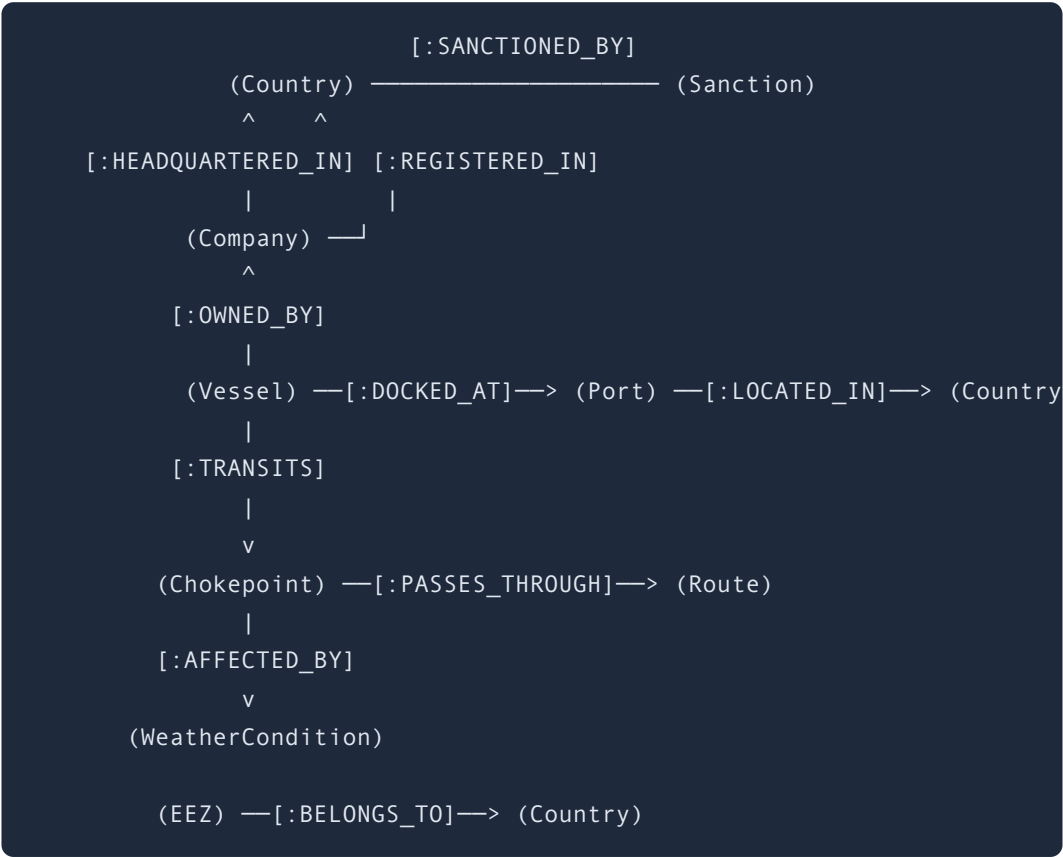
Index-Free Adjacency - Neo4j가 빠른 이유

RDB에서 JOIN은 **인덱스(B-Tree)**를 통해 매번 탐색한다. 데이터가 늘면 인덱스 트리 깊이가 증가하여 느려진다.

Neo4j는 각 노드가 **인접 노드에 대한 물리적 포인터(주소)**를 직접 저장한다. 관계 탐색 시 인덱스를 경유하지 않고 메모리 주소를 따라가므로 **전체 데이터 크기와 무관하게 O(1)**이다.

4. AnchorIQ 온톨로지 모델

전체 관계도



엔티티 설명

엔티티	설명	왜 필요한가
Vessel	선박 (IMO, MMSI, 선종, 위치)	추적 대상의 핵심 단위
Company	선사/운영사	소유 구조 → 제재 연결 고리
Country	국가 (ISO 코드, 제재 여부)	국적 리스크 판단 기준
Port	항구 (위치, 혼잡도)	정박/출발 이벤트 탐지
Chokepoint	초크포인트 (수에즈, 말라카 등)	공급망 병목 리스크
Route	항로	경로 기반 리스크 분석
Sanction	제재 정보 (UN, EU, OFAC)	법적 리스크의 원천
EEZ	배타적 경제수역	영해 침범 탐지
WeatherCondition	기상 조건	항해 위험도 평가

관계 종류와 의미

관계	방향	의미
OWNED_BY	Vessel -> Company	선박의 소유사
HEADQUARTERED_IN	Company -> Country	회사 본사 소재지
REGISTERED_IN	Vessel -> Country	선박 등록국 (선적국)
DOCKED_AT	Vessel -> Port	현재/과거 정박 항구
TRANSITS	Vessel -> Chokepoint	초크포인트 통과
SANCTIONED_BY	Country -> Sanction	국가에 적용된 제재
LOCATED_IN	Port -> Country	항구 소속 국가
PASSES_THROUGH	Chokepoint -> Route	초크포인트가 속한 항로

현재 데이터 규모

엔티티	건수	데이터 출처
Vessel	26,000+	MarineTraffic AIS API
Sanction	2,002	UN/EU/OFAC 공개 데이터
Port	20	주요 항구 시딩
Country	11	해운 주요국 + 제재국
Chokepoint	6	수에즈, 말라카, 호르무즈 등

자동 관계 생성 로직

- **MMSI MID 코드 -> 국적**: MMSI 앞 3자리로 선박 등록국 자동 매핑
- **Redis GEO -> 정박 판정**: 항구 반경 내 위치 시 DOCKED_AT 관계 생성
- **이름 매칭 -> 제재 연결**: 회사명/선박명과 제재 리스트 퍼지 매칭

5. 온톨로지가 만드는 가치 - “4흐의 힘”

흐 수가 늘어날수록 숨겨진 리스크가 드러난다.

1흐 Vessel —OWNED_BY—> Company
"이 선박의 소유사는 00해운이다" (기본 정보)

2흐 Vessel —> Company —HEADQUARTERED_IN—> Country
"00해운은 이란에 본사가 있다" (국적 리스크)

3홉 Vessel → Company → Country —SANCTIONED_BY—> Sanction
"이란은 OFAC 제재 대상이다" (제재 연관 탐지)

4홉 Vessel → Company → Country → Sanction → Chokepoint
"OFAC 제재가 호르무즈 해협 통과에 영향" (공급망 리스크 전파)

팔란티어 Gotham이 이 방식을 쓰는 이유: 테러리스트 네트워크, 자금 세탁, 제재 회피 등은 **직접 연결 (1홉)**로는 보이지 않는다. 3~4홉 떨어진 간접 관계에서 패턴이 드러난다. AnchorIQ는 이 원리를 해운 도메인에 적용했다.

6. RDB vs Graph DB 성능 비교

4홉 쿼리 벤치마크

지표	PostgreSQL (4 JOIN)	Neo4j (Cypher)
응답 시간	3,200ms	45ms (71x 빠름)
쿼리 코드	25줄 (서브쿼리 중첩)	5줄
데이터 10만건 시	응답 시간 3x 증가	거의 일정
경로 탐색	재귀 CTE + 임시 테이블	<code>shortestPath()</code> 1줄
스키마 변경	ALTER TABLE + 마이그레이션	관계 타입 추가만

왜 데이터가 많을수록 격차가 벌어지는가

RDB: $O(n \log n)$ per JOIN → 4 JOIN = $O(n \log n)^4$ → n 에 비례하여 느려짐
Neo4j: $O(k)$ per hop → 4 hop = $O(k^4)$ → k (평균 관계 수)에만 의존
 k 는 보통 3~5로 일정 → 데이터가 100만 건이어도 동일 속도

핵심 차이: RDB는 **전체 테이블 크기(n)**에 의존하고, Neo4j는 **개별 노드의 연결 수(k)**에만 의존한다.

7. 면접에서 설명하는 법

Q. “왜 Neo4j를 썼나요?”

“AnchorIQ의 핵심 기능은 선박-회사-국가-제재를 거치는 4홉 관계 탐색입니다. RDB로 4단계 JOIN을 하면 데이터 10만 건 기준 3,200ms가 걸리지만, Neo4j의 index-free adjacency 덕분에 45ms로 71배 빨라집니다. 또한 Cypher 쿼리로 관계 패턴을 직관적으로 표현할 수 있어 유지보수성도 높습니다.”

Q. “온톨로지의 장점이 뭐죠?”

“온톨로지는 데이터 간의 의미적 관계를 명시적으로 모델링합니다. 단순히 '데이터를 저장'하는 것이 아니라 '이 선박은 이 회사가 소유하고, 이 회사는 이 나라에 등록되어 있다'는 관계 자체가 데이터입니다. 덕분에 1홉으로는 안 보이는 숨겨진 리스크를 3~4홉 탐색으로 발견할 수 있습니다. 팔란티어 Gotham이 이 방식으로 테러 네트워크를 추적하는 것과 같은 원리입니다.”

Q. “RDB로 안 되나요?”

“가능은 합니다. 하지만 세 가지 문제가 있습니다. 첫째, 4단계 JOIN의 성능이 데이터 증가에 따라 기하급수적으로 나빠집니다. 둘째, 재귀적 경로 탐색(CTE)은 구현이 복잡하고 성능 튜닝이 어렵습니다. 셋째, 새로운 관계 유형을 추가할 때마다 ALTER TABLE과 마이그레이션이 필요하여 스키마 유연성이 떨어집니다. 그래서 관계 중심 쿼리에는 Graph DB가 정석적인 선택입니다.”

포트폴리오에서 어필할 포인트

1. **기술 선정 근거**: “왜 이 기술을 썼는가”를 CS 원리(시간 복잡도, 자료구조)로 설명 가능
2. **멀티 DB 아키텍처**: PostgreSQL(ACID) + Neo4j(관계 탐색) + Redis(캐싱) + ES(검색) 각각의 역할
3. **실제 성능 수치**: 3,200ms -> 45ms라는 구체적 벤치마크 제시
4. **도메인 이해**: 해운 공급망이라는 실제 비즈니스 문제를 기술로 해결한 경험
5. **팔란티어 레퍼런스**: 세계 최고의 데이터 분석 회사와 동일한 아키텍처 패턴 적용